



LECTURE

1) FINAL PROJECT

2) LAB ASSIGNMENT 1

3) GRAPH LLMS (G-RETRIEVER)

DATASCI290: NEUROSymbolic AI

JANUARY 28 2026



THE FINAL PROJECT



CARETRACE: THE TRUSTWORTHY MEDICAL TRIAGE AGENT

- Scenario: After-hours caregiver support for a child's symptoms
 - Time pressure, incomplete information, and strong desire to 'get through the night' safely
- Agent output must be **decisional** (not "encyclopedic")
 - A short 'tonight plan' + explicit 'go-now' thresholds + a clinician-style rationale
- Medicine is a flagship domain because **trust requirements are explicit**
 - The same architecture generalizes to finance, compliance, intelligence, emergency response ...

SCIENTIFIC MOTIVATION: WHAT WE ARE TESTING

- Observation
 - Frontier LLMs often respond prudently in medical Q&A
- Hypothesis
 - Trustworthy behavior is a system property: structured knowledge + executable practice logic + explicit uncertainty
- Empirical goal
 - Show where baselines (aka LLM as-is) drift (authority/context/state) versus
 - Where a Neurosymbolic pipeline stays disciplined

THIS IS AN INVESTIGATION !

- Can the LLM be **authoritative** ?
 - And thus, **trustworthy** ?
- Is a Neurosymbolic solution any better ?
 - Trustworthiness via authoritative, comprehensible, and thus actionable response

AUTHORITY AS THE CORE DIFFERENTIATOR (3 SOURCES)

- 1) Authoritative domain knowledge
 - Medication dosing for a specific formulation; contraindications; definitions/thresholds (e.g., dehydration signals)
 - Key point: SNOMED is for meaning; dosing comes from a drug reference table you treat as authoritative
- 2) Practice guidance
 - A scoped triage protocol: required questions, escalation triggers, exceptions
- 3) Judgment context
 - Local surveillance priors and test limitations; labeled as probabilistic context, never as certainty

NEUROSymbOLIC ARCHITECTURE

- Neural layer (LLM)
 - Interprets caregiver language into structured case fields; drives the dialogue
- Knowledge layer (mini KG)
 - Canonical concepts + IS_A hierarchy for generalization + related facts for evidence
- Symbolic layer (rules/constraints)
 - Executable practice logic that gates actions and emits a decision trace
- Context layer (priors)
 - Local surveillance feed used to prioritize questions and frame likelihoods (explicitly labeled)

WHAT A 'GOOD' RESPONSE LOOKS LIKE

- Short tonight plan
 - Hydration protocol + comfort measures + monitoring cadence
- Explicit go-now thresholds
 - A small set of red-flag triggers that override 'wait until morning' preferences
- Medication plan with provenance
 - Exact dose derived from: weight + formulation + named dosing reference; contraindication gates applied
- Clinician-style rationale
 - Key positives/negatives + which rule(s) fired + what's uncertain + what would change the plan

NESY PREVENTS 'INFORMATION BOMBARDMENT' BY DESIGN

- Conversation is a workflow, not free-form chat
 - Stage 1: screen red flags; Stage 2: stabilize; Stage 3: dose; Stage 4: safety-net plan
- Question selection is constrained
 - Ask only the next 2–4 high-value questions required by the protocol
- Progressive summarization
 - Maintain a compact case snapshot; update it rather than repeating everything

GENERATIVE AI/LLM COMPONENT

- Turns messy caregiver language into structured clinical facts (symptoms, timing, severity, meds, constraints) fast enough for real-time dialogue.
- Drives the interview: asks the next best question when required information is missing, and adapts to the caregiver's wording and anxiety.
- Produces a patient-friendly explanation, but only after it is “grounded” by the KG + checked by rules.
- Handles ambiguity explicitly (multiple candidate concepts with confidence), enabling the rest of the system to reason safely under uncertainty.

STRUCTURED KNOWLEDGE: KNOWLEDGE GRAPH COMPONENT

- Normalizes language to canonical clinical meaning (SNOMED concepts), making the system robust to paraphrase and synonym drift.
- Enables safe generalization via hierarchy (IS_A): rules can target categories (e.g., dehydration signs) rather than brittle strings.
- Supplies the minimal evidence bundle needed for decisions and explanations (relevant concepts + relationships), instead of dumping irrelevant context.
- Creates an auditable substrate: you can show “why” through explicit connections (concept → parent category → guideline trigger).

THE SYMBOLIC REASONING COMPONENT

- Encodes practice guidance as executable logic: obligations (must-ask), hard constraints (must-not-do), and escalation triggers.
- Provides deterministic action gating: prevents unsafe outputs that a generative model might produce under pressure or missing data.
- Produces a decision trace (which rules fired, which facts triggered them, what was blocked), enabling clinician-grade accountability.
- Enforces minimality: decides what to ask next and what to omit, preventing information overload and keeping the conversation decisional.

PRACTICE LOGIC: WHAT WE ENCODE (SCOPED RULES)

- Obligations (must-ask)
 - Weight before exact dosing; urine output before NSAID recommendation; alertness/breathing before 'stay home' advice
- Hard gates (must-not-do)
 - Block NSAID under dehydration risk; block dosing without formulation; block 'reassurance' if red flags unresolved
- Safety-netting thresholds
 - Escalation triggers: inability to keep fluids, worsening focal abdominal pain, prolonged anuria, altered mental status

COMPARISON

- LLM-only versus Neurosymbolic Agent (CareTrace)
- Will provide detailed criteria

FINAL PROJECT: FUNDAMENTAL OBJECTIVES

- That you will
 - Learn (something new and useful)
 - Create/Discover, (new) knowledge
 - Enjoy, the exercise



LAB ASSIGNMENT 1: LET'S CURATE A KNOWLEDGE GRAPH



LAB ASSIGNMENT: CURATING A “SMALL” KG

- Canonical meaning
 - Map paraphrases and messy language onto standard clinical concepts
- Hierarchy-driven generalization
 - Rules can target parent concepts instead of brittle strings
- Evidence for explanations
 - Use small, queryable subgraphs as justification artifacts





KG CURATION: PUBLIC SNOMED VIA SNOWSTORM → NEO4J AURADB

- Data access (public)
 - Snowstorm training endpoints: REST for concept search/details; FHIR \$lookup for labels
- Property graph model
 - (:Concept {sctid, pt})
 - (:Concept)-[:IS_A]->(:Concept)
 - (:Concept)-[:REL {typeId, typeTerm}]->(:Concept)
- Scope discipline
 - Seed 10–20 concepts; expand ancestors; add limited non-IS_A edges; cap total nodes

KG IN CARETRACE AGENT

- Normalization
 - LLM extraction → SNOMED concept IDs → stable features for rules
- Rule activation
 - Use IS_A to activate protocol logic at the right abstraction level
- Explainability
 - Return a minimal evidence bundle (concepts + relationships) alongside the decision trace

PROVIDED

- An accompanying Primer
- Accompanying code scaffold
- Encouragement: to use whatever AI based code generation tools/IDEs you would like to
 - Happy to provide recommendations as well



GRAPH RAG FOR TEXT + GRAPH UNDERSTANDING



THIS IS AN INTERESTING PAPER

- He, Xiaoxin, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. *G-retriever: Retrieval-augmented generation for textual graph understanding and question answering*. Advances in Neural Information Processing Systems 37 (2024): 132876-132907.
- Explains why standard vector-RAG fails on graphs and on global corpus questions.
 - Graphs need connected evidence; global questions need coverage/diversity, not a single “best chunk”.
- Designs a Graph-RAG pipeline end-to-end.
 - Index → retrieve → select/compose evidence → represent → generate → verify → evaluate.
- Know what to measure beyond answer accuracy.
 - Evidence validity/faithfulness; coverage/diversity; efficiency/token budget; robustness.

MOTIVATION: WHY 'JUST PROMPT THE LLM' IS BRITTLE

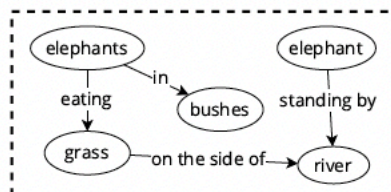
- LLMs are fluent but not guaranteed faithful.
 - When evidence is missing, they fill gaps with plausible text.
- Two failure modes
 - Local: invent nodes/edges/paths when graph context is incomplete.
 - Global: retrieve a few chunks and miss major themes or minority-but-critical signals.
- Graph RAG is a systems response
 - Make evidence retrieval + structure explicit; then generate from that evidence.

BASELINE: CLASSIC VECTOR RAG (AND ITS ASSUMPTIONS)

- Index-time
 - Chunk text → embed → store vectors + metadata.
- Query-time
 - Embed question → retrieve top-k chunks → prompt → generate.
- Where it breaks
 - Graph questions require connectivity; global questions require coverage.
 - Top-k retrieval is local and can be systematically biased.

GRAPH LLM

User



What kind of animal is in the bushes?

Answer the question and list the nodes or edges from the graph that support your answer.

LLM w/ Graph Prompt Tuning

[Incorrect response with hallucinated nodes and edges]

The animal in the bushes is a deer.

Nodes:

* Deer (node 1)

* Bushes (node 2)

Edges:

* Deer → Bushes (edge 1)

* Deer → Grass (edge 2)

* Bushes → Grass (edge 3)

G-Retriever (Ours)

[Correct response with accurate node and edge references]

Based on the information provided in the graph, the animal in the bushes is an elephant.

The nodes that support this answer are:

* Node 1: The **elephant** is mentioned in the first line of the graph.

The edges that support this answer are:

* The edge **connects the elephant to the bushes**, indicating that the elephant is in the bushes.

Therefore, the answer to the question is: The animal in the bushes is an elephant.

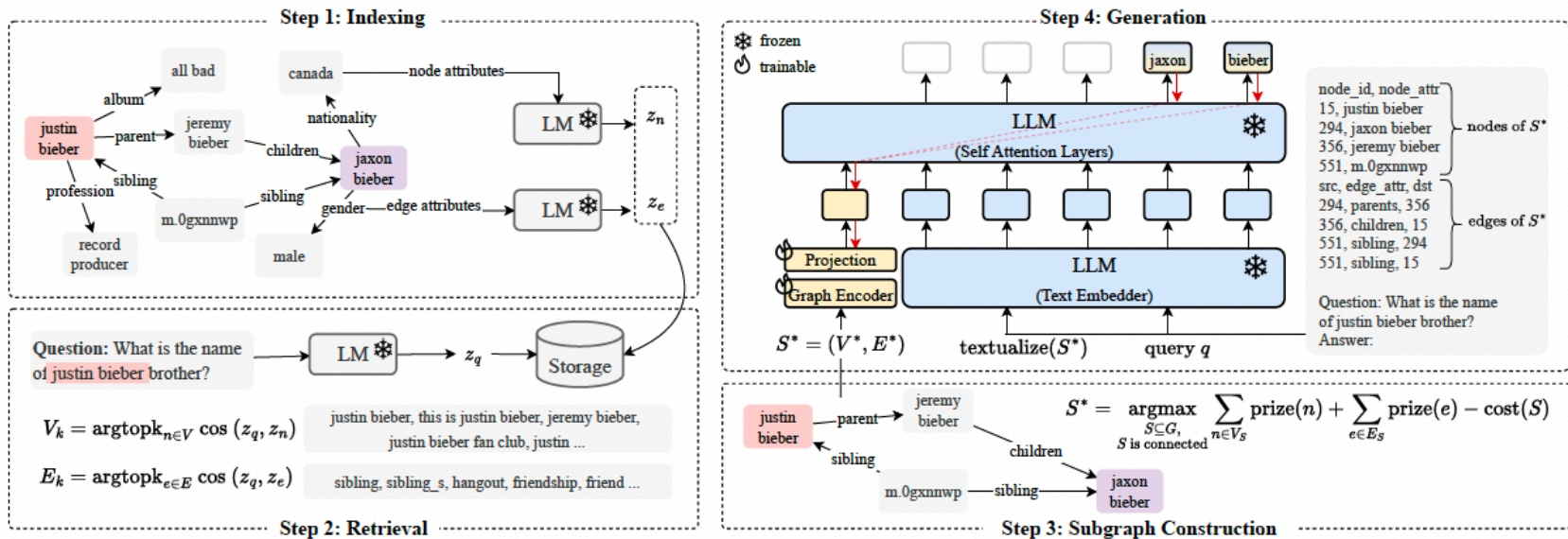
TWO WORKLOADS TO KEEP SEPARATE

- Workload A: QA over an existing graph
 - Graph is primary; questions require traversals/multi-hop relational evidence.
- Workload B: QA over a large corpus
 - Corpus is primary; global questions require synthesis across many chunks.
- Design implication
 - Different retrieval units, compression methods, and evaluation metrics.

GRAPH RAG: A UNIFYING MENTAL MODEL

- Core loop
 - Retrieve candidate evidence units.
 - Compose into compact evidence structure (subgraph or summary set).
 - Generate answer conditioned on evidence; optionally verify.
- Token budget is an algorithmic constraint
 - You are optimizing evidence selection under a hard context-length cap.

G-RETRIEVER: SYSTEM OVERVIEW (EVIDENCE SUBGRAPH RETRIEVAL)



- Large attributed graph (text on nodes/edges) + natural-language QA.
- **Pipeline**
 - Steps 1&2: Embed nodes + question → retrieve candidates.
 - Step 3: Select connected evidence subgraph via PCST.
 - Step 4: Represent subgraph for LLM (text + optional graph token) → generate answer.

STEP 1: GRAPH INDEXING

STEP 2: RETRIEVAL

- Node textualization
 - Each node i has text t_i ; embed to vector e_i .
- Question embedding
 - Embed question q to e_q ; $\text{similarity}(e_q, e_i)$ yields node relevance ('prize').
- Scale considerations
 - Use ANN index for node embeddings; cache embeddings; consider hybrid lexical+vector retrieval.

WHY CONNECTIVITY MATTERS (BRIDGE-NODE PHENOMENON)

- Multi-hop evidence is common
 - Intermediate entities are required to justify answers.
- Disconnected top-k failure
 - Retrieves endpoints (A, C) but misses bridge (B) → LLM invents B or invents $A \rightarrow C$.
- Connected evidence subgraph
 - Includes B and edges; answer can cite an explicit path.

STEP 3: SUBGRAPH SELECTION

- Inputs

- Candidate nodes with relevance scores.
- Graph edges with costs (penalties) for inclusion.

- Goal

- Choose a small connected subgraph maximizing: total relevance – total cost.

- Why optimization

- Heuristics (top-k + k-hop) often pull noise or still miss connectors.

PCST: PRIZE-COLLECTING STEINER TREE

- PCST intuition: Collect valuable nodes (prizes) while paying edge costs to connect them.
- Mapping:
 - Prize = relevance to query
 - Cost = penalty for extra structure (proxy for token budget/noise).
- Result: Compact connected evidence (often tree-like) that links key nodes via minimal connectors

$$S^* = \operatorname{argmax}_{\substack{S \subseteq G, \\ S \text{ is connected}}} \sum_{n \in V_S} \text{prize}(n) + \sum_{e \in E_S} \text{prize}(e) - \text{cost}(S),$$

STEP 4: ANSWER GENERATION (LLM + A “GRAPH TOKEN” + THE TEXTUALIZED SUBGRAPH)

- Retrieved evidence subgraph S^* is fed to the model in two channels: a compact structure signal and an explicit text evidence view.
- Graph token (soft prompt): a graph neural network encodes S^* into a single pooled vector; that vector is projected into the LLM’s embedding space and inserted as an extra “token” before the text.
- Textualized subgraph (evidence): the same S^* is also serialized into a compact list of facts (nodes + labeled edges, CSV-like) and concatenated with the user question as normal text input.
- Frozen LLM generation: the LLM’s weights are not updated; it generates the answer conditioned on both inputs: (1) the graph token embedding and (2) the textualized evidence + question embeddings.
- Training updates only the added components (graph encoder + projection), which makes adaptation lightweight while still letting the LLM leverage both graph structure and explicit facts.

EXAMPLE: DRUG–TARGET–PATHWAY–PHENOTYPE

- Question
 - Why might Drug D cause adverse event E?
- Evidence you want
 - D targets T; T participates in pathway P; P associated with phenotype E.
- Failure mode
 - If T/P missing from evidence, model invents mechanism or confuses correlated concepts.
- Graph RAG fix
 - Connected evidence subgraph makes mechanism explicit and citable.

EVALUATION BEYOND ACCURACY: EVIDENCE VALIDITY

- Why
 - A plausible answer can be wrong if it relies on invented nodes/edges.
- Metrics
 - Valid nodes: mentioned nodes exist in graph.
 - Valid edges: mentioned edges exist in graph.
 - Fully valid subgraph: all elements are valid and consistent.
- Takeaway: Graph QA allows hard faithfulness checks via membership tests in the KG.